Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-212БВ-25

Студент: Минин А.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 29.09.26

Москва, 2026

Постановка задачи

Вариант 11.

Родительский процесс создает два дочерних процесса. Перенаправление стандартных потоков ввода-вывода показано на картинке выше. Child1 и Child2 можно «соединить» между собой дополнительным каналом. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процесс child1 и child2 производят работу над строками. Child2 пересылает результат своей работы родительскому процессу. Родительский процесс полученный результат выводит в стандартный поток вывода. Child1 переводит строки в верхний регистр. Child2 превращает все пробельные символы в символ «_».

Общий метод и алгоритм решения

Использованные системные вызовы:

- pipe(int[2]); – Создание трёх неименованных каналов: pipe1 (parent → child1), pipe_between (child1 → child2), pipe2 (child2 → parent);
- fork(); – Создание двух дочерних процессов: child1 и child2;
- dup2(old, new); - Перенаправление стандартных потоков ввода-вывода в дочерних процессах на нужные концы каналов;
- close(fd); - Закрытие неиспользуемых концов каналов (обязательно для корректного EOF и предотвращения утечек дескрипторов);
- execv(path, argv); - Замена образа дочернего процесса на исполняемый файл;
- read(fd, buf, n); - Чтение данных из канала или со стандартного ввода;
- write(fd, buf, n); - Запись данных в канал, в стандартный вывод или в стандартный поток ошибок;
- waitpid(pid, &status, 0); - Ожидание завершения дочерних процессов и получение их статуса (кода возврата или номера сигнала);
- exit(code); - Завершение процесса с кодом возврата.

Программа реализует конвейерную обработку строк: пользователь вводит строку, она проходит через два дочерних процесса, каждый из которых выполняет свою трансформацию, и возвращается родителю для вывода.

1. Создание каналов (родитель). Родитель вызывает pipe() три раза и получает шесть файловых дескрипторов:
   - pipe1[0] (чтение) / pipe1[1] (запись) — от родителя к child1;
   - pipe_between[0] / pipe_between[1] — от child1 к child2;
   - pipe2[0] / pipe2[1] — от child2 к родителю.
2. Создание child1. Родитель вызывает fork(). В дочерней ветке:
   - закрывает ненужные концы каналов (pipe1[1], pipe_between[0], pipe2[0], pipe2[1]);
   - через dup2(pipe1[0], STDIN_FILENO) подменяет стандартный ввод на чтение из pipe1;
   - через dup2(pipe_between[1], STDOUT_FILENO) подменяет стандартный вывод на запись в pipe_between;
   - вызывает execv("./child1", ...), чтобы заменить свой образ на программу child1.

После execv процесс child1 «не знает», что работает с каналами — для него это обычные stdin и stdout.

3. Создание child2. Аналогично, но с другими концами каналов:
   - dup2(pipe_between[0], STDIN_FILENO) — вход из pipe_between;
   - dup2(pipe2[1], STDOUT_FILENO) — выход в pipe2;
   - execv("./child2", ...).
4. Родитель после fork. Закрывает все концы каналов, которые ему не нужны:
   - pipe1[0] (читает child1),
   - pipe_between[0], pipe_between[1] (в этом канале родитель не участвует),
   - pipe2[1] (пишет child2).

Остаются только pipe1[1] (запись) и pipe2[0] (чтение).

5. Основной цикл родителя.
   - Родитель читает строку от пользователя через read(STDIN_FILENO, ...).
   - Если строка пустая (один символ \n) — выходит из цикла.
   - Иначе пишет её в pipe1[1].
   - Затем читает ответ из pipe2[0] и выводит его в STDOUT_FILENO.
   - Проверяет полноту каждой операции: written != bytes или bytes < 0 → сообщение об ошибке в stderr и выход.
6. Работа child1.
   - Читает порции до 4096 байт из stdin (это pipe1).
   - Переводит каждый символ в верхний регистр через toupper.
   - Пишет результат в stdout (это pipe_between).
7. Работа child2.
   - Читает из stdin (это pipe_between).
   - Заменяет все пробельные символы (' ', '\t', '\n', '\r', '\v', '\f') на _.
   - Пишет в stdout (это pipe2).
8. Завершение. Когда пользователь ввёл пустую строку, родитель:
   - закрывает pipe1[1] — child1 получает EOF на read и завершается;
   - закрывает pipe2[0] — child2 не может больше писать, а после EOF из pipe_between (закрытого child1) тоже завершается;
   - вызывает wait(NULL) и ждёт завершения дочерних процессов;
   - выходит с кодом 0.

Код программы

parent.c

```
#include <stdint.h>
#include <stdbool.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <signal.h>

static ssize_t print_uint(int fd, uint64_t n) {
    char buf[32];
    size_t i = 0;

    if (n == 0) {
        char zero = '0';
        return write(fd, &zero, 1);
    }

    while (n > 0) {
        buf[i++] = (char)('0' + (n % 10));
        n /= 10;
    }

    for (size_t j = 0; j < i / 2; ++j) {
        char tmp = buf[j];
        buf[j] = buf[i - 1 - j];
        buf[i - 1 - j] = tmp;
    }

    return write(fd, buf, i);
}

static char CHILD1_PROGRAM[] = "child1";
static char CHILD2_PROGRAM[] = "child2";

int main(void) {
    int pipe1[2];
    int pipe_between[2];
    int pipe2[2];

    if (pipe(pipe1) == -1) {
        const char msg[] = "error: failed to create pipe1\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
    if (pipe(pipe_between) == -1) {
        const char msg[] = "error: failed to create pipe_between\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
    if (pipe(pipe2) == -1) {
        const char msg[] = "error: failed to create pipe2\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    pid_t child1 = fork();
    if (child1 == -1) {
        const char msg[] = "error: fork child1 failed\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    if (child1 == 0) {
        close(pipe1[1]);
        close(pipe_between[0]);
        close(pipe2[0]);
        close(pipe2[1]);

        if (dup2(pipe1[0], STDIN_FILENO) == -1) {
            const char msg[] = "error: dup2 child1 stdin failed\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
        }
        close(pipe1[0]);

        if (dup2(pipe_between[1], STDOUT_FILENO) == -1) {
            const char msg[] = "error: dup2 child1 stdout failed\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
```

```
        exit(EXIT_FAILURE);
    }
    close(pipe_between[1]);

    char *const args[] = {CHILD1_PROGRAM, NULL};
    execv(CHILD1_PROGRAM, args);

    const char msg[] = "error: failed to exec child1\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

pid_t child2 = fork();
if (child2 == -1) {
    const char msg[] = "error: fork child2 failed\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    waitpid(child1, NULL, 0);
    exit(EXIT_FAILURE);
}

if (child2 == 0) {
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe_between[1]);
    close(pipe2[0]);

    if (dup2(pipe_between[0], STDIN_FILENO) == -1) {
        const char msg[] = "error: dup2 child2 stdin failed\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
    close(pipe_between[0]);

    if (dup2(pipe2[1], STDOUT_FILENO) == -1) {
        const char msg[] = "error: dup2 child2 stdout failed\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
    close(pipe2[1]);

    char *const args[] = {CHILD2_PROGRAM, NULL};
    execv(CHILD2_PROGRAM, args);

    const char msg[] = "error: failed to exec child2\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

close(pipe1[0]);
close(pipe_between[0]);
close(pipe_between[1]);
close(pipe2[1]);

{
    const char msg[] = "Parent started.\nChild1 PID: ";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    print_uint(STDOUT_FILENO, child1);
    const char msg2[] = "\nChild2 PID: ";
    write(STDOUT_FILENO, msg2, sizeof(msg2) - 1);
    print_uint(STDOUT_FILENO, child2);
    const char msg3[] = "\nType text to get started (empty line to exit):\n";
    write(STDOUT_FILENO, msg3, sizeof(msg3) - 1);
}

char buf[4096];
ssize_t bytes;

while (1) {
    bytes = read(STDIN_FILENO, buf, sizeof(buf));
    if (bytes < 0) {
        const char msg[] = "error: parent failed to read stdin\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        break;
    }
    if (bytes == 0 || buf[0] == '\n') break;

    ssize_t written = write(pipe1[1], buf, bytes);
    if (written != bytes) {
        const char msg[] = "error: parent failed to write to pipe1 (data limited to 4096 bytes)\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        break;
    }
```

```
        bytes = read(pipe2[0], buf, sizeof(buf));
        if (bytes < 0) {
            const char msg[] = "error: parent failed to read from pipe2\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            break;
        }

        if (bytes > 0) {
            {
                const char msg[] = "Result: ";
                write(STDOUT_FILENO, msg, sizeof(msg) - 1);
            }

            written = write(STDOUT_FILENO, buf, (size_t)bytes);
            if (written != bytes) {
                const char msg[] = "error: parent failed to write to stdout (data limited to 4096 bytes)\n";
                write(STDERR_FILENO, msg, sizeof(msg) - 1);
                break;
            }

            {
                const char msg[] = "\n";
                write(STDOUT_FILENO, msg, sizeof(msg) - 1);
            }
        }
    }

    close(pipe1[1]);
    close(pipe2[0]);

    pid_t exitd1 = wait(NULL);
    {
        print_uint(STDOUT_FILENO, exitd1);
        const char msg[] = " exited.\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    }
    pid_t exitd2 = wait(NULL);
    {
        print_uint(STDOUT_FILENO, exitd2);
        const char msg[] = " exited.\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    }

    {
        const char msg[] = "Parent exited.\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    }

    return 0;
}
```

child1.c

```
#include <stdint.h>
#include <stdbool.h>
#include <unistd.h>
#include <stdlib.h>
#include <ctype.h>

int main(void) {
   char buf[4096];
   ssize_t bytes;

   while ((bytes = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
      for (uint32_t i = 0; i < bytes; ++i) {
         buf[i] = toupper(buf[i]);
      }

      ssize_t written = write(STDOUT_FILENO, buf, (size_t)bytes);
      if (written != bytes) {
         const char msg[] = "error: child1 failed to write.\n";
         write(STDERR_FILENO, msg, sizeof(msg) - 1);
         exit(EXIT_FAILURE);
      }
```

```
    }

    if (bytes < 0) {
        const char msg[] = "error: child1 failed to read.\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    return 0;
}
```

child2.c

```
#include <stdint.h>
#include <stdbool.h>
#include <unistd.h>
#include <stdlib.h>
#include <ctype.h>

int main(void) {
    char buf[4096];
    ssize_t bytes;

    while ((bytes = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
        for (uint32_t i = 0; i < bytes; ++i) {
            if (isspace((unsigned char)buf[i])) {
                buf[i] = '_';
            }
        }

        ssize_t written = write(STDOUT_FILENO, buf, (size_t)bytes);
        if (written != bytes) {
            const char msg[] = "error: child2 failed to write.\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
        }
    }

    if (bytes < 0) {
        const char msg[] = "error: child2 failed to read.\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    return 0;
}
```

Протокол работы программы

Тестирование:

```
Parent started.
Child1 PID: 41937
Child2 PID: 41938
Type text to get started (empty line to exit):
foo
Result: FOO_
foo bi
Result: FOO_BI__
12 334  4
Result: 12_334_4_

41937 exited.
41938 exited.
```

Parent exited.

Вывод

Программа реализует конвейерную обработку строк через два дочерних процесса, связанных неименованными каналами: родитель читает ввод пользователя и передаёт его через pipe1 в child1, который переводит символы в верхний регистр, после чего данные через pipe_between попадают в child2, где все пробельные символы заменяются на _, и результат через pipe2 возвращается родителю для вывода на экран. Все взаимодействия выполняются исключительно через системные вызовы pipe, fork, dup2, execv, read, write, close, waitpid и wait, без использования обёрток стандартной библиотеки. Перенаправление стандартных потоков через dup2 позволяет дочерним программам работать с обычными stdin/stdout, не зная о каналах, а проверка полноты операций ввода-вывода (written != bytes или bytes < 0) гарантирует корректную обработку ошибок с ограничением буфера в 4096 байт. При завершении родитель аккуратно закрывает каналы, собирает статусы детей через wait.